

159.235 Week 1: Java and IntelliJ

Lecture Material

- Introduction to Java
- Drawing on a graphical device

The main aim for the first week is to ensure you are all set up to use Java and can run some Java basic and drawing programs.

Review Questions

Introduction to Java

1. What is Java? What is required to make Java code run on a given OS?
2. What is the Java Virtual machine? What are its components and what is its role in running Java programs?
3. What are important differences between Java and C/C++ ?
4. What does it mean to say that Java is *strictly* and *statically* typed?
5. How does Java deal with scoping of variables?
6. What Java operators are there? How do they compare with C/C++ ?
7. How does Java deal with flow control?
8. How does pass-by-value compare with pass by reference?
9. What is a Java class and what do you do with them?
10. What are Java exceptions? How are they handled? How can you define your own exceptions?
11. How does Java handle these important concepts in object-oriented programming: overloading, class constructors, inheritance, and polymorphism?
12. What is a Java *interface*?
13. What is the difference between an *abstract* class and a *concrete* class?

Drawing on a graphics device

14. What are some examples of graphics devices?
15. What role does a graphics card play in the computer memory model of a graphics device?
16. What is the principle behind *bitmap* graphics devices? How does this compare with the old style *vector map* graphics? What are some examples of each?
17. How are *rectilinear* coordinate systems implemented on graphics display devices?

18. What does it mean to talk about *graphics primitives*? What are some examples?
19. What do the following coordinate transformations do: translation, rotation, scale, shear?
20. What is *clipping* and why is it important in computer graphics?

Exercises

Getting started with IntelliJ

1. If you are in one of the computer labs on campus, find a terminal and log yourself in. If working remotely, visit <https://remote.massey.ac.nz>. Either log in via the Web interface or (better) install the VMWare for your system.
2. Open IntelliJ from the Windows start menu under JetBrains.
3. Create a new project from the menu **File**→**New**→**Project**. Choose a name for your project (eg **BasicDemos**) and create it. Where is the project folder stored on the system?
4. Have a look at the left hand side project side panel and note how your project folder is structured. Your Java source code will go under **src**.
5. Open the **Main.java** program. Run it by pushing the green play button. You can find this on the top toolbar or by right clicking inside the code display editing area. What is the result of running this program? How does IntelliJ run it?
6. Make a new package (e.g. **nz.ac.massey.basic**) and copy the “Java Demos Basic Demos” from Stream into this new folder. What changes do you need to make to each of these Java files in order to make them work inside the package.
7. Run these demos and study how they work by considering what was discussed in the lectures.

Installing IntelliJ Yourself

You can skip this section if you are only going to use the Massey computer lab machines.

To run IntelliJ on your own system, you should visit the “Getting Started” tutorial on the JetBrains site:

<https://www.jetbrains.com/help/idea/getting-started.html>

1. Verify that your system meets at least the minimum requirements. Preferably it should meet the recommended requirements.
2. Follow the instructions for “standalone” installation. If you are not applying for a student license, then make sure to download the Community Edition.
3. Follow the installation wizard on your system. Note:
 - i. Don’t install any additional plugins; they won’t be needed for this course.

- ii. Unless you have a previous version of IntelliJ installed, don't import settings
 - iii. The dark theme is recommended—but use light if you prefer
 - iv. You will be prompted to use an existing JDK on your system, or download from JetBrains.
 - v. Packages are a major part of Java projects. If you use packages, consider standard naming conventions, e.g.
 - nz.ac.massey**
 - nz.ac.massey.basic**
 - nz.ac.massey.toolbox**
 - etc.
4. Try the above exercises on your own installation of IntelliJ.
 5. Have a look at the other tutorial material on the JetBrains site.

Running Java from the command line

This exercise is optional. You can compile and run Java programs directly from the command line outside of the IntelliJ IDE. You will still need to have the Java SDK and JRE installed on your system.

Instructions below are for Windows OS. For MacOSX and Linux, just replace the backslash directory separator \ with a forward slash /.

1. Make a folder tree on your system **nz\ac\massey\basic**.
2. Copy a Java program (e.g. **Main.java**) into the bottom **basic** sub-directory. Make sure the package is correctly set in **Main.java**.
3. Compile the program. Issue this command from the top of the directory tree

```
javac nz\ac\massey\basic\Main.java
```

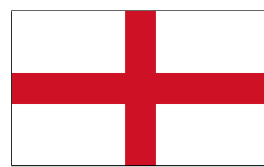
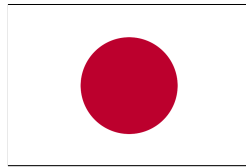
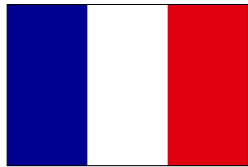
4. Run the program

```
java nz.ac.massey.basic.Main
```

Drawing with Java

1. Using IntelliJ, either create a new project or a new package within an existing project. Copy the demo programs from Stream under “Demos: drawing”.
2. Run **DemoDraw01.java**. You should see a little red square on a yellow background. Examine the Java code and figure out what is going on. In particular:
 - i. What containers are created and how are they being used?
 - ii. How does this program make use of inheritance?
 - iii. How is the coordinate system defined?
3. Run **DemoDraw02.java**. Note the usage of the Java **Color** class.

- i. Try different colours and examine the separate R, G, and B values. Compare with the `argb` integer words.
 - ii. For the alpha compositing, try experimenting with different colours and α values
4. Run `DemoDraw03.java`. Note the various primitives that appear and study the corresponding code.
5. Run `DrawDemo04.java` and `DrawDemo05.java`. Note the different transformations on show here. Also note how the transformations are implemented by re-defining the coordinate system—this is the *passive* view.
6. Run `DrawDemo06.java`. Look in the implementation of the `paintComponent()` method.
 - i. Uncomment the 3 lines of code after “A rectangular clipping region” and run again. What do you see happening here?
 - ii. Re-comment these 3 lines and uncomment the line after “A circular clipping region”. Run the program again. Now what happens?
7. Write a new program that will draw the flags of France, Japan, and England as below?



Hint: for the Japanese flag, consider using the `g2.fillOval()` method. Also the colour of the disc has RGB(207,8,31)—more crimson than red.